L'Architecture Client-Serveur et le Protocole HTTP

Thème principal : Fonctionnement du Web, Protocole HTTP, Architecture Client-Serveur et Codes de Statut

Synthèse exécutive : Ce cours pose les fondements indispensables du Web en détaillant l'architecture client-serveur asymétrique et le fonctionnement du protocole HTTP (HyperText Transfer Protocol). L'étudiant y découvre comment un navigateur (client) formule des requêtes textuelles pour obtenir des ressources auprès d'un serveur web, comment ce dernier y répond de manière asynchrone et sans état (stateless), la structure interne des messages HTTP (lignes de commande/statut, en-têtes et corps), la différenciation stricte entre les méthodes GET et POST, ainsi que le décodage systématique des codes de statut (2xx, 3xx, 4xx, 5xx) indispensables au débogage d'applications web.

1. L'Architecture Client-Serveur et le Principe Stateless

L'architecture client-serveur est le modèle fondamental de communication sur le Web. Contrairement aux réseaux Peer-to-Peer (P2P) où chaque nœud est équivalent, ce modèle impose une asymétrie stricte des rôles : le client (le navigateur) initie toujours la communication, tandis que le serveur attend passivement. Une caractéristique majeure du protocole HTTP est d'être 'stateless' (sans état). Le serveur n'enregistre aucune mémoire de la transaction précédente. Dès que la réponse HTTP est transmise au client, la connexion est rompue ou remise à zéro, et le serveur oublie l'identité de l'utilisateur. Si ce principe garantit une excellente scalabilité des serveurs web (qui n'ont pas à conserver des millions d'états en mémoire), il interdit nativement la gestion d'un panier d'achat ou le maintien d'une session utilisateur connectée. Pour contourner cette contrainte, on utilise des cookies transmis automatiquement dans les en-têtes de chaque requête HTTP.

- Client (Navigateur) : Logiciel actif qui prend toujours l'initiative d'envoyer une requête pour obtenir une ressource.
- Serveur Web : Machine ou processus logiciel passif (ex: Apache, Nginx) qui écoute sur un port (80 pour HTTP, 443 pour HTTPS) et répond aux demandes.
- Protocole Stateless (Sans état) : Absolutisme du protocole HTTP où chaque transaction requête-réponse est totalement isolée et amnésique.
- Mécanismes de Session : Emploi de cookies (côté client) et de sessions (côté serveur) pour maintenir un état persistant (panier d'achat, connexion).

Cas concret enseignant : *Le professeur compare HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un parfait inconnu. Pour qu'il se souvienne de vous, vous devez lui présenter un badge (le cookie de session) à chaque phrase. De plus, lors d'une visite sur le site de l'IUT de Bayonne, le navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer la page HTML puis chaque image, fichier CSS et script JS.*

2. Structure des Messages HTTP et Méthodes (GET vs POST)

Toute requête HTTP est un bloc de texte structuré de façon immuable : 1. La ligne de requête (Méthode, URI, Version HTTP) ; 2. Les en-têtes HTTP (metadata comme Host ou User-Agent) ; 3. Une ligne vide obligatoire faisant office de séparateur strict ; 4. Le corps (Body) optionnel. La réponse du serveur adopte la même mécanique mais débute par la ligne de statut (Version, Code numérique, Phrase d'explication). Le choix de la méthode conditionne la sécurité et le comportement de la demande. La méthode GET est dite 'sûre' et 'idempotente' car elle ne doit pas modifier l'état des données du serveur. Ses variables sont transmises directement dans l'URI (ex: ?recherche=informatique). À l'inverse, POST transporte ses données dans le corps de la requête, ce qui permet d'envoyer des volumes importants (fichiers, images) et de masquer les variables dans la barre d'adresse.

- Message Textuel : Un échange HTTP est entièrement rédigé en texte clair lisible à l'œil nu.
- Ligne de Requête / Statut : Première ligne définissant la méthode, l'URI et le protocole (requête) ou la version et le code de statut (réponse).

• En-têtes (Headers) : Paires clé-valeur fournissant des métadonnées contextuelles (Host, User-Agent, Content-Type).

• Méthode GET : Conçue pour la simple consultation de ressources. Les paramètres transitent de manière visible dans l'URL.

• Méthode POST : Conçue pour soumettre des données modifiant l'état du serveur. Les paramètres sont encapsulés dans le corps (body) de la requête.

Cas concret enseignant : *Le professeur insiste sur le fait que l'utilisation de la méthode GET pour un formulaire de connexion ou d'inscription est une faille critique. Transmettre un mot de passe via GET l'affiche directement dans l'URL, le stocke dans l'historique du navigateur, dans les logs des serveurs mandataires, et l'expose aux yeux de toute personne passant derrière l'écran ('shoulder surfing').*

3. Classification des Codes de Statut HTTP

Chaque réponse HTTP renvoie obligatoirement un code de statut numérique à trois chiffres réparti en cinq grandes classes. Pour le développeur web, la compréhension de la frontière entre 4xx et 5xx est capitale lors de la phase de débogage : les codes 4xx indiquent que la responsabilité incombe au client (URL mal tapée, accès interdit, ressource déplacée sans redirection). Le correctif doit être apporté côté client ou sur l'URL ciblée. Les codes 5xx signalent quant à eux un plantage du serveur (erreur de syntaxe dans un script PHP/Node.js, base de données inaccessible, saturation mémoire). La responsabilité réside exclusivement sur l'infrastructure ou le code backend.

• 2xx (Succès) : Requête reçue, comprise et traitée avec succès (ex: 200 OK).

• 3xx (Redirection) : Actions supplémentaires requises de la part du client (ex: 301 Moved Permanently).

• 4xx (Erreur Client) : Anomalie provenant de la requête formulée par le client (ex: 404 Not Found, 403 Forbidden).

• 5xx (Erreur Serveur) : Défaillance interne du serveur qui a échoué à traiter une demande valide (ex: 500 Internal Server Error).

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, il avait mal configuré un script PHP dont la connexion à la base de données échouait lors de la validation d'un formulaire, renvoyant systématiquement une erreur 500. Les utilisateurs effrayés pensaient que le site avait été piraté, alors qu'il s'agissait simplement d'un point-virgule manquant dans son code backend.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : Avec la méthode GET, les identifiants et mots de passe sont concaténés à la fin de l'URL sous la forme '?login=matei&password;=1234'. Ils deviennent visibles par tout le monde dans la barre d'adresse, restent enregistrés dans l'historique du navigateur et apparaissent en clair dans les fichiers de logs du serveur web. Avec POST, les identifiants sont placés dans le corps (body) du message HTTP, de manière invisible dans la barre d'adresse. Attention toutefois : POST masque la donnée dans l'URL mais ne la chiffre pas ! Seul l'emploi conjoint de HTTPS permet de chiffrer l'intégralité de la transaction sur le réseau.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante. Identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer le comportement du serveur avec le code de retour exact si la ressource est absente : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Ligne de requête ('GET /images/logo.png HTTP/1.1') : Le premier terme indique la méthode HTTP. Il s'agit de GET (demande de consultation/récupération d'une ressource sans altération de l'état du serveur). La ressource sollicitée (URI) est '/images/logo.png'. Le protocole utilisé est HTTP version 1.1. 2. En-tête Host ('Host: iutbayonne.univ-pau.fr') : Cet en-tête obligatoire en HTTP/1.1 spécifie le nom de domaine du serveur cible, indispensable pour le virtual hosting (plusieurs sites hébergés sur la même IP). 3. En-tête User-Agent ('User-Agent: Mozilla/5.0') : Identifie le logiciel client (navigateur) qui émet la requête. 4. Scénario d'absence de la ressource : Si

le fichier 'logo.png' ne se trouve pas dans le dossier '/images/' du serveur web, le serveur ne peut exécuter la requête. L'erreur étant due à une demande de ressource inexistante par le client, le serveur doit renvoyer une erreur de la classe 4xx, précisément le code '404 Not Found'.

Résultat : Méthode : GET | Ressource : /images/logo.png | Hôte cible : iutbayonne.univ-pau.fr | Code d'erreur en cas d'absence : 404 Not Found (Erreur Client).

■ Auto-Évaluation Flash d'Examen

Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP et quelle est sa conséquence sur la gestion des sessions ?

R : Un protocole 'stateless' (sans état) ne conserve aucune mémoire des requêtes précédentes : chaque échange est indépendant et le serveur oublie le client immédiatement après la réponse. Pour gérer une session (connexion, panier), il faut utiliser des mécanismes externes comme des cookies.

Q : Quelle est la différence fondamentale de responsabilité entre un code de statut 4xx et un code de statut 5xx ?

R : Un code 4xx désigne une erreur imputable au client (URL incorrecte, ressource inexistante, manque d'autorisation), alors qu'un code 5xx désigne une défaillance interne du serveur (crash de script, base de données inaccessible) sur une requête pourtant valide.

Q : Pourquoi la méthode HTTP GET ne doit-elle jamais être employée pour transmettre un mot de passe ?

R : Parce que GET transmet les paramètres directement dans l'URL. Les données confidentielles apparaissent alors en clair dans la barre d'adresse, sont enregistrées dans l'historique du navigateur et dans les fichiers de logs du serveur, exposant l'utilisateur au 'shoulder surfing'.